VIVA GUIDE 01 / 05

The student-facing experience

React, Vite, interaction design, and the complete question-to-answer journey.

YOUR RESPONSIBILITY

Own the browser interface: how a student asks, follows, and revisits a question.

THE SHORT VERSION

The frontend is a React single-page app. It presents conversation history, submits questions to the Express API, shows loading and service readiness, and renders answers with their source links. Its job is to make the underlying document-grounded system understandable and usable.

A 30-SECOND OPENING

I own the student experience. The React app keeps the interface focused on asking a question, reading the answer, and checking the official pages behind it. It also makes service availability visible instead of hiding setup problems.

IN THIS GUIDE

- Purpose and role
- The page and its state
- Question-to-answer interaction
- Citations and honest status
- Build, run, and demonstrate
- Limitations and likely viva questions

PURPOSE AND ROLE

The client is the student's entry point to the NIT Hamirpur information assistant. It is built with React and Vite, with the main experience in client/src/App.jsx and its visual rules in client/src/index.css. It does not call Gemini directly: it calls the Express API through the shared Axios client.

- Collect a question or let a student start from a suggested question.
- Show recent conversations and let the student create, reopen, or delete a session.
- Display the answer, its source documents and page numbers, and connection/readiness status.
- Keep the experience usable on desktop and a narrow mobile screen.

The boundary matters: the browser renders sources supplied by the backend; it does not decide which institute document is authoritative.

THE PAGE AND ITS STATE

App.jsx stores session list, selected session, messages, the input draft, loading state, a user-visible error, sidebar state, backend connection state, and AI readiness in React state. Refs are used for the message-list end marker so new messages can scroll into view.

On mount, one effect loads saved sessions and selects the first available session. A second checks /health to set the connection and readiness labels. A third scrolls to the latest message when messages or the loading indicator change. These effects keep initial page setup separate from the send interaction.

- An empty conversation shows four suggested questions rather than an empty chat transcript.
- A non-empty conversation maps each saved message to a role-labelled article and renders message text with ReactMarkdown.
- The sidebar contains session controls; on smaller screens it is an overlay with a backdrop and close action.

QUESTION-TO-ANSWER INTERACTION

```
sendQuestion(text)
trim and reject blank / duplicate sends
clear draft; show loading state
create a session if none is selected
append an optimistic user message
POST /chat { sessionId, message }
append answer + sources from response
refresh session and sidebar list
clear loading state
```

The optimistic user message appears before the network response, so the interface feels responsive. The loading indicator then tells the student the service is working. If the request fails at the client/network boundary, an alert explains that the service is unavailable. The input is disabled while a request is running, preventing accidental duplicate submissions.

The send button submits the form, and Enter submits unless Shift is held. A visually-hidden label, accessible button names, role=status for status updates, role=alert for errors, aria-live for the transcript, and aria-current for the selected conversation help assistive-technology users.

CITATIONS AND HONEST STATUS

Each assistant message can contain a sources array. The UI shows the document title and page. When a URL

exists it opens in a new tab with rel=noopener noreferrer; without a URL, the source is displayed as plain text rather than a broken link.

The top-right status is based on /health. It distinguishes a reachable backend from Gemini readiness and whether the document index is ready. This is useful because the web server can be healthy while the AI key is missing or the NIT index has not been generated.

- No API key is placed in browser code. Gemini configuration stays in the server-side AI service.
- The interface explains that answers are based on cited documents; it does not present an uncited answer as an official institute decision.
- The status is a snapshot from page load; this version does not continuously poll health.

BUILD, RUN, AND DEMONSTRATE

From the client directory, install the declared npm dependencies, then run npm run dev for local development or npm run build to create the production bundle. The Axios base URL is configured in client/src/services/api.js and defaults to the local backend API. Follow the repository README for the complete multi-service startup order.

- Open the app and point out the empty state, suggested questions, connection label, and recent-conversations panel.
- Ask a question covered by the hostel booklet or fee structure. Explain the loading state and then open one cited source.
- Create a second conversation, switch between the two, and delete one to show session controls.
- Show how missing Gemini configuration appears in readiness instead of exposing a key in the browser.

Viva phrasing: React owns presentation and interaction state. Persistent conversation data and AI answers come from the API; keeping that boundary avoids duplicating server logic in the browser.

LIMITATIONS AND LIKELY VIVA QUESTIONS

The browser currently uses a single main component rather than a larger component hierarchy. Health is checked once at startup, and session operations depend on a reachable API. Markdown is rendered from the server answer, so a future deployment should review its allowed render behavior and add robust request cancellation/retry UX where needed.

Why does the frontend call Express instead of Gemini?
It keeps the API key out of the browser and gives the system one place to manage sessions, validation, and AI-service communication.

Why show citations beside the answer?
They let the student verify the answer against the official document and the cited page.

What is an optimistic message?
A temporary UI update that shows the user's question immediately, before the server request completes.

What happens if there is no session yet?
The client creates one through POST /sessions before sending the chat request.

How does the app adapt to mobile?
The conversation sidebar becomes a toggleable drawer with a backdrop and explicit close controls.

Does the client save conversations?

No. It requests session data from Express, which currently persists it in local JSON files.